

Synth-pop Music Generation — Data Pipeline

CSE 153 Final Project

This notebook builds a curated synth-pop MIDI dataset from the **Lakh MIDI Dataset (LMD)** for two music-generation tasks:

- 1. **Task 1 — Symbolic, Unconditioned Generation:** Generate synth-pop *melodies* from scratch.
- 2. **Task 2 — Symbolic, Conditioned Generation:** Generate *drum patterns* conditioned on BPM, style, and a 2-bar seed.

This notebook covers the **data side** of the pipeline: collecting, filtering, extracting, and validating ~340 synth-pop songs. Track-extraction utilities and the final dataset are then handed off via `src/data_utils.py` so the modeling notebooks stay clean.

Pipeline overview

Stage	What we do	Output
1. Setup	Locate the LMD Clean MIDI subset on disk	<code>DATA_DIR</code>
2. Filter to synth-pop	Match a curated artist list against LMD	384 candidate songs
3. Inspect	Manually examine one song to understand MIDI structure	—
4. Quality survey	Check BPM and drum availability across the subset	374 valid-tempo, 368 with drums
5. Build <i>usable</i> set	Drop songs that fail quality filters	<code>usable_songs.csv</code>
6. Task 1: Extract melodies	Heuristic-based melody-track detection (iterated 3 versions)	335 melody MIDIs
7. Task 2: Extract drums	Channel-9 mechanical extraction	340 drum MIDIs
8. Hand-off	Package everything into a reusable module	<code>src/data_utils.py</code>

1. Setup & data location

Project paths. The LMD Clean MIDI subset (~17,000 MIDI files organized by artist) is expected at `data/clean_midi/`. We use absolute paths so the notebook can be

opened from any directory without `..` confusion.

```
In [1]: import sys
        from pathlib import Path

        # — Raw LMD data: stays in the old Assignment 2 folder (it's the source, &
        DATA_DIR = Path('/Users/donghyunhahn/Desktop/spring 2026/cse 153/Assignment

        # — Repo root: the notebook lives in <repo>/data_extraction/, so go up one
        REPO_ROOT = Path.cwd().parent # data_extraction/ -> synth_music_generator/
        PROCESSED_DIR = REPO_ROOT / 'processed'
        PROCESSED_DIR.mkdir(exist_ok=True)

        sys.path.insert(0, str(REPO_ROOT))

        print(f"Raw data:      {DATA_DIR}")
        print(f"  exists:      {DATA_DIR.exists()}")
        print(f"Repo root:      {REPO_ROOT}")
        print(f"Processed dir: {PROCESSED_DIR}")

        if DATA_DIR.exists():
            n = len([d for d in DATA_DIR.iterdir() if d.is_dir()])
            print(f"\n✓ Found {n} artist folders in raw data")
        else:
            print("\n⚠ Raw data not found at that path!")
```

```
Raw data:      /Users/donghyunhahn/Desktop/spring 2026/cse 153/Assignment 2/
data/clean_midi
  exists:      True
Repo root:     /Users/donghyunhahn/Documents/GitHub/synth_music_generator
Processed dir: /Users/donghyunhahn/Documents/GitHub/synth_music_generator/pr
ocessed
```

✓ Found 2198 artist folders in raw data

Standard scientific Python stack plus `pretty_midi` for MIDI I/O and
`IPython.display` for inline audio playback.

```
In [2]: import pretty_midi
        import numpy as np
        import pandas as pd
        import matplotlib.pyplot as plt
        from IPython.display import Audio, display
        from tqdm import tqdm
        import warnings
        warnings.filterwarnings('ignore')
```

```
/Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/site-packa
ges/pretty_midi/instrument.py:11: UserWarning: pkg_resources is deprecated a
s an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The p
kg_resources package is slated for removal as early as 2025-11-30. Refrain f
rom using this package or pin to Setuptools<81.
  import pkg_resources
```

2. Exploring the LMD

LMD Clean MIDI is organized as `clean_midi/<Artist Name>/<Song>.mid`. First, how many artists are in the dataset? We expect around 900, but the Clean MIDI release contains roughly 2,200 (including duplicates and variant spellings).

```
In [3]: artists = sorted([d.name for d in DATA_DIR.iterdir() if d.is_dir()])
print(f"Total artists: {len(artist)}\n")
print("First 30 artists alphabetically:")
for a in artists[:30]:
    print(f" {a}")
```

Total artists: 2198

First 30 artists alphabetically:

```
"Weird Al" Yankovic
.38 Special
10,000 Maniacs
101 Strings
10cc
1910 Fruitgum Company
2 Brothers on the 4th Floor
2 Unlimited
2 the Core
20 Fingers
2Boys
311
3T
4 Non Blondes
702
808 State
883
911
98 Degrees
A Taste of Honey
A*Teens
ABBA
ABC
AC DC
AL Bano
ANKA PAUL
ARMSTRONG LOUIS
Aaliyah
Aaron Neville
Ace
```

3. Filtering to synth-pop

We curate a list of synth-pop artists spanning three eras:

- **Classic 80s** (a-ha, Depeche Mode, Pet Shop Boys, Erasure, Eurythmics, ...)
- **90s electronic-leaning pop** (Garbage, Saint Etienne)

- **Modern synth-pop / synthwave** (CHVRCHES, M83, The 1975)

We then do case-insensitive matching against LMD's artist folders and count how many songs we have per artist. This determines whether the dataset is large enough to train on.

```
In [4]: SYNTHPOP_ARTISTS = [
    # Classic 80s synth-pop
    'a-ha', 'Depeche Mode', 'Tears for Fears', 'Duran Duran',
    'New Order', 'Pet Shop Boys', 'OMD', 'Eurythmics',
    'Human League', 'The Human League', 'Erasure', 'Yazoo',
    'Soft Cell', 'Ultravox', 'Howard Jones', 'Thompson Twins',
    'Wham!', 'ABC', 'Talk Talk', 'Spandau Ballet',
    'Visage', 'Heaven 17', 'Bronski Beat',
    'Frankie Goes to Hollywood', 'Culture Club',
    'Simple Minds', 'INXS', 'The Cars',
    # 90s/2000s electronic-leaning pop
    'Saint Etienne', 'Garbage',
    # Modern synth-pop
    'CHVRCHES', 'M83', 'The 1975', 'Chromatics',
    'Future Islands', 'Passion Pit', 'Phoenix',
    'Empire of the Sun', 'La Roux', 'Goldfrapp',
    'Ladyhawke', 'Robyn',
]

available_lower = {a.lower(): a for a in artists}

found = []
not_found = []
for target in SYNTHPOP_ARTISTS:
    if target.lower() in available_lower:
        found.append(available_lower[target.lower()])
    else:
        not_found.append(target)

found = sorted(set(found))

print(f"Found in LMD: {len(found)}/{len(SYNTHPOP_ARTISTS)}\n")

total_songs = 0
print("✓ Found artists:")
for a in found:
    songs = list((DATA_DIR / a).glob('*.mid')) + list((DATA_DIR / a).glob('*.mp3'))
    print(f"  {a}: {len(songs)} songs")
    total_songs += len(songs)

print(f"\n✗ Not in LMD:")
for a in not_found:
    print(f"  {a}")

print(f"\n📊 Total synth-pop songs available: {total_songs}")
```

Found in LMD: 28/42

✓ Found artists:

- ABC: 3 songs
- Bronski Beat: 1 songs
- Culture Club: 4 songs
- Depeche Mode: 68 songs
- Duran Duran: 41 songs
- Erasure: 42 songs
- Eurythmics: 26 songs
- Frankie Goes to Hollywood: 9 songs
- Garbage: 26 songs
- Howard Jones: 4 songs
- INXS: 13 songs
- New Order: 8 songs
- OMD: 2 songs
- Pet Shop Boys: 44 songs
- Robyn: 1 songs
- Simple Minds: 11 songs
- Soft Cell: 4 songs
- Spandau Ballet: 6 songs
- Talk Talk: 2 songs
- Tears for Fears: 14 songs
- The Cars: 25 songs
- The Human League: 4 songs
- Thompson Twins: 2 songs
- Ultravox: 1 songs
- Visage: 2 songs
- Wham!: 8 songs
- Yazoo: 2 songs
- a-ha: 11 songs

x Not in LMD:

- Human League
- Heaven 17
- Saint Etienne
- CHVRCHES
- M83
- The 1975
- Chromatics
- Future Islands
- Passion Pit
- Phoenix
- Empire of the Sun
- La Roux
- Goldfrapp
- Ladyhawke

 Total synth-pop songs available: 384

4. Anatomy of a synth-pop MIDI

Before writing any extraction code, we need to **understand what a multi-track MIDI actually looks like**. Different songs have different track layouts — some have 4 tracks,

others have 20 — and track names are inconsistent ("MELODY", "Lead", "Vocal", "Track 1", or just blank).

We pick one well-known song (Depeche Mode — "A Question of Lust") and dump its full structure. This gives us a baseline mental model for the extraction heuristic later.

```
In [5]: # Pick the first Depeche Mode song
test_artist = 'Depeche Mode'
artist_dir = DATA_DIR / test_artist
songs = sorted(list(artist_dir.glob('*.mid')) + list(artist_dir.glob('*.midi')))

print(f"Songs by {test_artist}:")
for i, s in enumerate(songs[:10]):
    print(f"  [{i}] {s.name}")

# Pick song index 0 (or change to pick a specific one)
test_file = songs[0]
print(f"\nLoading: {test_file.name}")

midi = pretty_midi.PrettyMIDI(str(test_file))

print(f"\nDuration:      {midi.get_end_time():.1f} seconds")
print(f"Tempo:            {midi.estimate_tempo():.1f} BPM")
print(f"Time signature: {midi.time_signature_changes}")
print(f"Total tracks:    {len(midi.instruments)}")
```

Songs by Depeche Mode:

```
[0] A Question of Lust.mid
[1] Any Second Now (Voices).mid
[2] Barrel Of A Gun.mid
[3] Big Muff.mid
[4] Black Celebration.1.mid
[5] Black Celebration.2.mid
[6] Black Celebration.3.mid
[7] Black Celebration.mid
[8] Blasphemous Rumours.1.mid
[9] Blasphemous Rumours.2.mid
```

Loading: A Question of Lust.mid

```
Duration:      288.8 seconds
Tempo:         142.3 BPM
Time signature: [TimeSignature(numerator=4, denominator=4, time=0.0)]
Total tracks:  7
```

Now print every track with its instrument program, note count, and average pitch.

Already we can guess which track is the melody: it'll have a moderate note count (not too sparse, not arpeggiated), an average pitch in the vocal range (MIDI 55-80), and ideally a name-hint like "Lead" or "Synth String".

```
In [6]: # DETAILED TRACK BREAKDOWN
print(f"\n{'#':<3} {'Instrument':<32} {'Notes':<7} {'AvgPitch':<10} {'IsDrum':<10}")
print("-" * 90)
```

```

for i, inst in enumerate(midi.instruments):
    if inst.is_drum:
        instr_name = "DRUMS"
    else:
        instr_name = pretty_midi.program_to_instrument_name(inst.program)

    pitches = [n.pitch for n in inst.notes]
    avg_pitch = np.mean(pitches) if pitches else 0
    track_name = (inst.name[:23] if inst.name else "(unnamed)")

    print(f"{i:<3} {instr_name:<32} {len(inst.notes):<7} {avg_pitch:<10.1f}
          {str(inst.is_drum):<8} {track_name}")

```

#	Instrument	Notes	AvgPitch	IsDrum	Name
0	Synth Bass 1	147	57.1	False	(unnamed)
1	FX 4 (atmosphere)	256	38.2	False	(unnamed)
2	FX 4 (atmosphere)	162	38.5	False	(unnamed)
3	String Ensemble 1	136	56.3	False	(unnamed)
4	Synth Strings 2	125	72.4	False	(unnamed)
5	Pad 3 (polysynth)	2	70.0	False	(unnamed)
6	String Ensemble 1	107	36.4	False	(unnamed)

Piano-roll visualization

Showing each track as a piano roll makes the structure obvious. We can visually identify:

- **Bass** — sustained low pitches with repeating patterns
- **Melody / lead** — moderate density, mid-high pitches, melodic contour
- **Pads / strings** — sustained chords, mid-range
- **Drums** — `is_drum=True`, no pitch information visible

```

In [7]: # VISUALIZE ALL TRACKS (PIANO ROLL)
n_tracks = len(midi.instruments)
fig, axes = plt.subplots(n_tracks, 1, figsize=(15, 2 * n_tracks), sharex=True)

if n_tracks == 1:
    axes = [axes]

for i, (inst, ax) in enumerate(zip(midi.instruments, axes)):
    piano_roll = inst.get_piano_roll(fs=20)

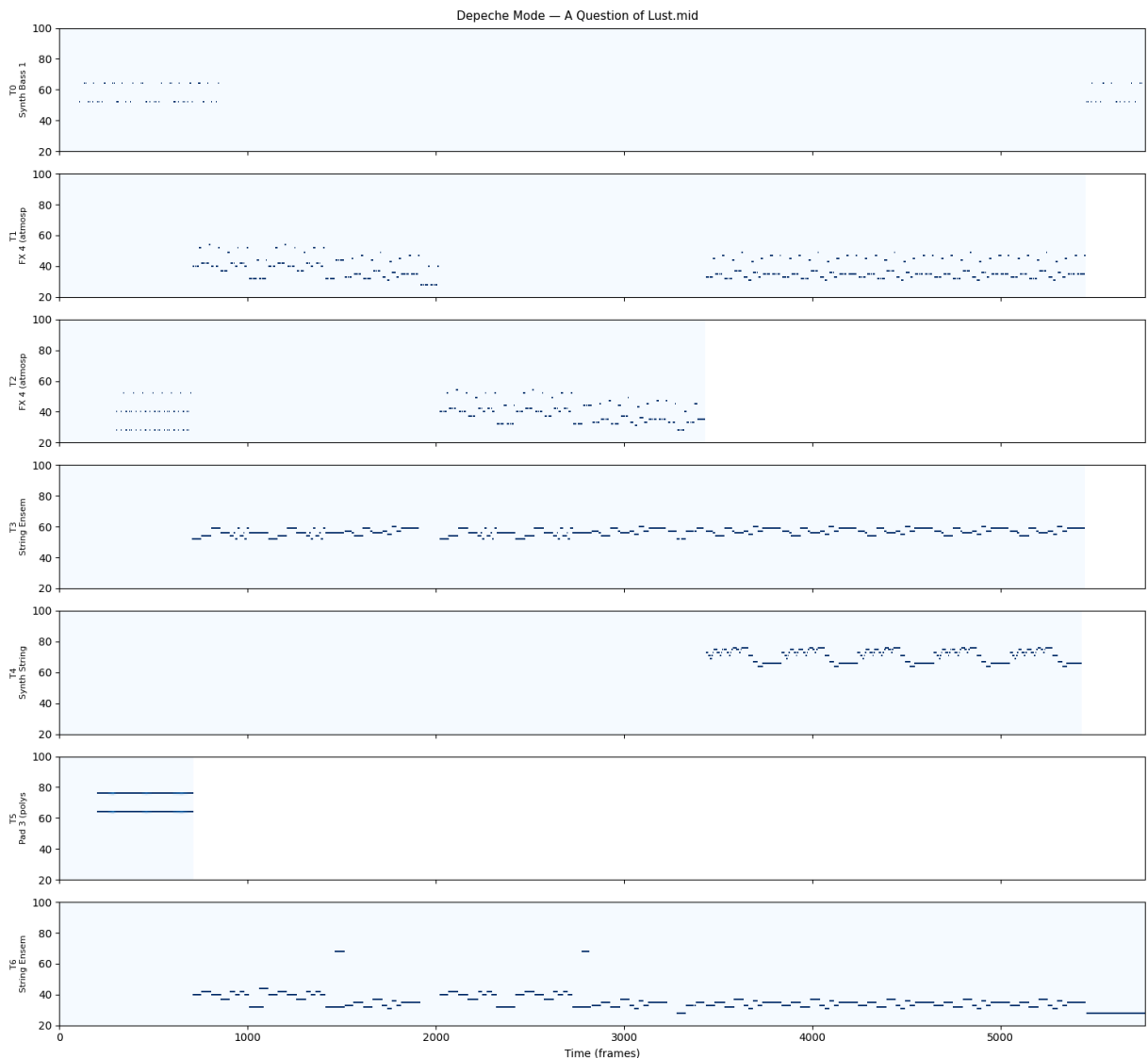
    if inst.is_drum:
        label = "DRUMS"
    else:
        label = pretty_midi.program_to_instrument_name(inst.program)

    ax.imshow(piano_roll, aspect='auto', origin='lower',
              cmap='Blues', interpolation='nearest')
    ax.set_ylabel(f"T{i}\n{label[:12]}", fontsize=8)
    ax.set_ylim(20, 100)

axes[-1].set_xlabel('Time (frames)')

```

```
plt.suptitle(f'{test_artist} - {test_file.name}', fontsize=11)
plt.tight_layout()
plt.show()
```



5. BPM survey — and a bug we caught

We want to know the BPM distribution of synth-pop. The naive approach is

`midi.estimate_tempo()` — but **this method often returns half-time or double-time values** depending on which beat subdivision the estimator latches onto. A 100 BPM song with lots of 16th notes can be reported as 200 BPM.

The fix is to use the MIDI file's *stored* tempo events (set explicitly by whoever transcribed the MIDI) via `midi.get_tempo_changes()`. We define a helper for this, then run it on the full synth-pop subset.

```
In [8]: def get_tempo(midi):
        """Get tempo from MIDI's tempo events (more reliable than estimate)."""
        tempo_changes = midi.get_tempo_changes()
        # tempo_changes is (times, tempos); take the first or median
```



```

if len(tempo_changes[1]) > 0:
    return float(np.median(tempo_changes[1]))
# Fallback: estimate (but this is unreliable)
return midi.estimate_tempo()

```

Running BPM extraction across all ~384 songs (takes ~30s). The histogram should peak near **120 BPM** — the textbook synth-pop tempo. If it peaks at 240, our `get_tempo` function is still being fooled.

```

In [9]: def get_tempo(midi):
        """Get tempo from MIDI's tempo events (more reliable than estimate)."""
        times, tempos = midi.get_tempo_changes()
        if len(tempos) > 0:
            return float(np.median(tempos))
        return midi.estimate_tempo() # fallback

print("Resampling BPMs using stored tempo events...")

bpms = []
for artist in tqdm(found, desc="Artists"):
    artist_dir = DATA_DIR / artist
    songs = list(artist_dir.glob('*.mid')) + list(artist_dir.glob('*.midi'))
    for song in songs:
        try:
            m = pretty_midi.PrettyMIDI(str(song))
            tempo = get_tempo(m)
            if 40 < tempo < 220:
                bpms.append((artist, song.name, tempo))
        except Exception:
            continue

bpm_df = pd.DataFrame(bpms, columns=['artist', 'song', 'bpm'])
print(f"\nTotal songs with valid tempo: {len(bpm_df)}")
print(f"BPM range: {bpm_df['bpm'].min():.0f} - {bpm_df['bpm'].max():.0f}")
print(f"Median BPM: {bpm_df['bpm'].median():.0f}")

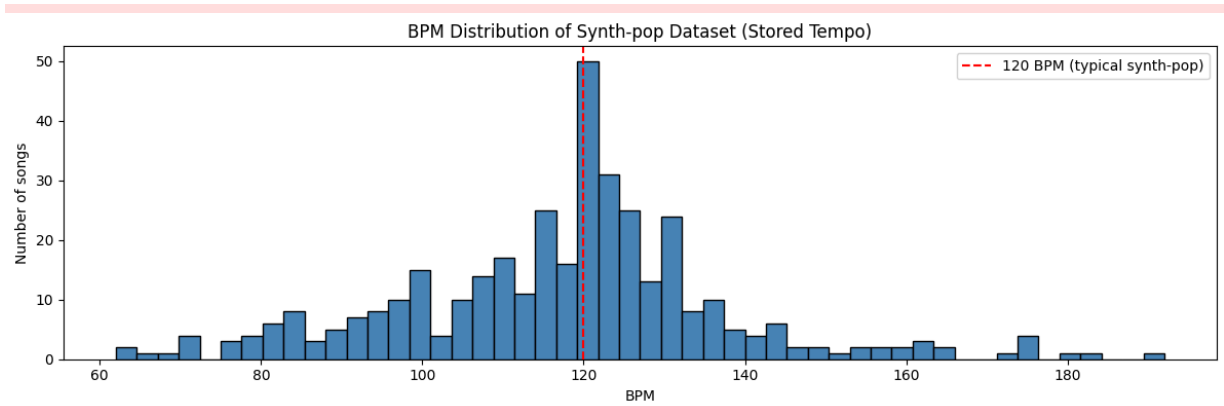
fig, ax = plt.subplots(figsize=(12, 4))
ax.hist(bpm_df['bpm'], bins=50, color='steelblue', edgecolor='black')
ax.axvline(120, color='red', linestyle='--', label='120 BPM (typical synth-p)')
ax.set_xlabel('BPM')
ax.set_ylabel('Number of songs')
ax.set_title('BPM Distribution of Synth-pop Dataset (Stored Tempo)')
ax.legend()
plt.tight_layout()
plt.show()

print("\nMedian BPM by artist (sorted):")
artist_bpm = bpm_df.groupby('artist')['bpm'].agg(['count', 'median', 'std'])
print(artist_bpm.sort_values('median'))

```

Resampling BPMs using stored tempo events...

Artists: 100%|██████████| 28/28 [00:17<00:00, 1.62it/s]
 Total songs with valid tempo: 374
 BPM range: 62 - 192
 Median BPM: 120



Median BPM by artist (sorted):

	count	median	std
artist			
Ultravox	1	72.0	NaN
Robyn	1	92.0	NaN
Tears for Fears	14	99.5	16.1
Spandau Ballet	6	100.0	22.5
Simple Minds	11	103.0	9.2
Yazoo	2	110.0	25.5
Wham!	8	112.5	35.2
Visage	2	113.5	2.1
Erasure	42	114.0	17.5
Frankie Goes to Hollywood	9	116.0	19.6
Howard Jones	4	116.0	6.0
Culture Club	4	117.0	43.5
Duran Duran	41	117.0	17.9
The Human League	2	119.0	1.4
Garbage	24	120.0	17.7
Depeche Mode	66	120.0	19.4
INXS	13	120.0	16.9
ABC	3	121.0	19.5
Pet Shop Boys	43	121.0	11.0
OMD	2	121.3	29.3
Thompson Twins	2	122.3	81.0
Eurythmics	24	124.0	12.0
Talk Talk	2	124.0	1.4
Bronski Beat	1	127.0	NaN
New Order	8	128.3	24.4
The Cars	25	130.0	22.6
Soft Cell	4	142.0	26.8
a-ha	10	160.0	28.4

6. Drum availability check

Task 2 (drum generation) requires drum tracks. But how many synth-pop MIDIs actually *have* drums? Some early Depeche Mode tracks are sparse and might lack a drum machine track.

Drums in MIDI are mechanically identifiable: channel 9 is reserved for drums, and `pretty_midi` exposes this via `inst.is_drum`. We count drum availability across the full subset and check that we have enough data for Task 2.

```
In [10]: print("Checking how many songs have drum tracks...")

drum_stats = []
for artist in tqdm(found, desc="Artists"):
    artist_dir = DATA_DIR / artist
    songs = list(artist_dir.glob('*.mid')) + list(artist_dir.glob('*.midi'))
    for song in songs:
        try:
            m = pretty_midi.PrettyMIDI(str(song))

            # Find drum tracks
            drum_tracks = [inst for inst in m.instruments if inst.is_drum]
            n_drum_tracks = len(drum_tracks)
            n_drum_notes = sum(len(inst.notes) for inst in drum_tracks)

            drum_stats.append({
                'artist': artist,
                'song': song.name,
                'has_drums': n_drum_tracks > 0,
                'n_drum_tracks': n_drum_tracks,
                'n_drum_notes': n_drum_notes,
                'n_total_tracks': len(m.instruments),
            })
        except Exception:
            continue

drum_df = pd.DataFrame(drum_stats)

print(f"\nTotal songs analyzed: {len(drum_df)}")
print(f"Songs WITH drums:      {drum_df['has_drums'].sum()} ({100*drum_df['has_drums'].sum()/len(drum_df)}%)")
print(f"Songs WITHOUT drums:    {(~drum_df['has_drums']).sum()} ({100*(~drum_df['has_drums'].sum()/len(drum_df))}%)")

# Songs with substantial drum content (>50 notes)
substantial = drum_df[drum_df['n_drum_notes'] >= 50]
print(f"Songs with substantial drums (50+ notes): {len(substantial)}")

# Per-artist breakdown
print("\nDrum availability by artist:")
artist_drums = drum_df.groupby('artist').agg(
    total_songs=('song', 'count'),
    songs_with_drums=('has_drums', 'sum'),
```

```
.assign(pct=lambda x: (100 * x['songs_with_drums'] / x['total_songs']).round(1))
print(artist_drums.sort_values('songs_with_drums', ascending=False))
```

Checking how many songs have drum tracks...

Artists: 100%|██████████| 28/28 [00:17<00:00, 1.58it/s]

Total songs analyzed: 377

Songs WITH drums: 368 (97.6%)

Songs WITHOUT drums: 9 (2.4%)

Songs with substantial drums (50+ notes): 361

Drum availability by artist:

	total_songs	songs_with_drums	pct
artist			
Depeche Mode	67	60	90.0
Pet Shop Boys	43	43	100.0
Erasure	42	42	100.0
Duran Duran	41	40	98.0
The Cars	25	25	100.0
Eurythmics	24	24	100.0
Garbage	24	23	96.0
Tears for Fears	14	14	100.0
INXS	13	13	100.0
Simple Minds	11	11	100.0
a-ha	10	10	100.0
Frankie Goes to Hollywood	9	9	100.0
New Order	8	8	100.0
Wham!	8	8	100.0
Spandau Ballet	6	6	100.0
The Human League	4	4	100.0
Culture Club	4	4	100.0
Soft Cell	4	4	100.0
Howard Jones	4	4	100.0
ABC	3	3	100.0
Talk Talk	2	2	100.0
Thompson Twins	2	2	100.0
Visage	2	2	100.0
OMD	2	2	100.0
Yazoo	2	2	100.0
Bronski Beat	1	1	100.0
Ultravox	1	1	100.0
Robyn	1	1	100.0

7. Persist metadata

Save the BPM and drum-availability findings to CSV so we don't have to re-process 384 MIDI files each session. We also merge them into a single `song_metadata.csv` that becomes the source of truth for the rest of the pipeline.

```
In [11]: # Save BPM data
bpm_df.to_csv(PROCESSED_DIR / 'bpm_data.csv', index=False)

# Save drum data
drum_df.to_csv(PROCESSED_DIR / 'drum_data.csv', index=False)
```

```
# Merge into one master metadata file
master_df = bpm_df.merge(
    drum_df[['artist', 'song', 'has_drums', 'n_drum_notes', 'n_total_tracks']
    on=['artist', 'song'], how='outer'
)
master_df.to_csv(PROCESSED_DIR / 'song_metadata.csv', index=False)

print(f"Saved {len(master_df)} rows to {PROCESSED_DIR / 'song_metadata.csv'}")
print(f"\nColumns: {list(master_df.columns)}")
print(f"\nPreview:")
print(master_df.head(10))
```

Saved 377 rows to /Users/donghyunhahn/Documents/GitHub/synth_music_generator/processed/song_metadata.csv

Columns: ['artist', 'song', 'bpm', 'has_drums', 'n_drum_notes', 'n_total_tracks']

Preview:

	artist	song	bpm	has_drums
0	ABC	4 Ever 2 Gether.mid	89.999955	True
1	ABC	Look of Love, Part 1.mid	120.999944	True
2	ABC	Poison Arrow.mid	126.000126	True
3	Bronski Beat	Small Town Boy.mid	126.999985	True
4	Culture Club	Church of the Poison Mind.mid	130.000130	True
5	Culture Club	Do You Really Want to Hurt Me.1.mid	82.000007	True
6	Culture Club	Do You Really Want to Hurt Me.mid	104.000014	True
7	Culture Club	Karma Chameleon.mid	183.004941	True
8	Depeche Mode	A Question of Lust.mid	94.999992	False
9	Depeche Mode	Any Second Now (Voices).mid	172.000103	True

	n_drum_notes	n_total_tracks
0	3214	11
1	1714	16
2	2263	14
3	2931	12
4	2533	14
5	1538	16
6	1538	16
7	2982	15
8	0	7
9	48	12

Quality filters

Apply a few sanity filters to narrow `master_df` down to the **usable** subset:

- Must have drums (for Task 2)
- At least 50 drum notes (some songs have a "drum" track with one trigger hit)
- BPM between 70 and 160 (excludes outliers and any remaining tempo-estimation glitches)

Also attach the absolute filepath to each row so downstream code can `pretty_midi.PrettyMIDI(row.filepath)` directly.

```
In [12]: # A clean list of usable songs (has drums, valid tempo, reasonable BPM)
usable = master_df[
    (master_df['has_drums'] == True) &
    (master_df['n_drum_notes'] >= 50) &
    (master_df['bpm'] >= 70) &
    (master_df['bpm'] <= 160)
].copy()

# Add the full filepath for easy loading
def make_filepath(row):
    return str(DATA_DIR / row['artist'] / row['song'])

usable['filepath'] = usable.apply(make_filepath, axis=1)

usable.to_csv(PROCESSED_DIR / 'usable_songs.csv', index=False)

print(f"

```

 Filtered dataset:

Original songs:	377
After quality filters:	340
Loss:	37 songs (9.8%)

By artist:

artist	
Depeche Mode	58
Pet Shop Boys	42
Erasure	42
Duran Duran	36
Eurythmics	24
The Cars	24
Garbage	21
Tears for Fears	13
Simple Minds	11
INXS	10
Frankie Goes to Hollywood	8
New Order	7
Wham!	6
Spandau Ballet	6
a-ha	5
Soft Cell	4
Howard Jones	4
Culture Club	3
ABC	3
Talk Talk	2
OMD	2
The Human League	2
Visage	2
Yazoo	2
Robyn	1
Bronski Beat	1
Ultravox	1

dtype: int64

8. Task 1: Melody extraction

This is the hardest single piece of the pipeline. Synth-pop MIDI files don't label tracks consistently — sometimes the lead is called "Vocal", sometimes "Lead", sometimes "Synth String 4", and sometimes just (unnamed) . We need a heuristic that identifies the melody track from track features alone.

Iteration history

This function went through three versions, each driven by listening to the extracted outputs:

Version	Approach	Problem found
v1	Pitch range + monophony + name hint	Picked a 34-note Vibraphone over the iconic Tenor Sax in "Careless Whisper" — perfect monophony beat a real melody
v2	Added instrument-program bonuses	Still underweighted note density
v3	Added minimum-note-count gate, density bonus, broader "lead-like" program list	Used below. Works for ~87% of songs

The fundamental insight: **a sparse, perfectly-monophonic track is rarely the melody.** Real melodies are dense (100–500 notes for a 3-min song), played on lead instruments, and tolerate some note overlap (sax/voice legato).

Scoring breakdown in v3

- **Hard requirements:** ≥ 50 notes, in pitch range C3–C6, $\geq 40\%$ monophonic, song ≥ 30 s long
- **Soft bonuses:** lead instrument (+0.6), known-good melody instrument (+0.15), name match (+0.3 to +0.5), density 0.5–3.5 notes/sec (+0.2), pitch range 7–30 semitones (+0.15)
- **Penalties:** too dense > 3.5 notes/sec $\rightarrow$ likely arpeggio (–0.1)

```
In [13]: def find_melody_track_v3(midi):
        """
        v3: Smarter scoring that prioritizes note density and lead instruments.
        Lessons from v1/v2:
        - Vibraphone with 34 notes is not melody (need note count requirement)
        - Tenor Sax with sustained notes shouldn't lose to perfectly monophonic
        - Synth-pop lead instruments matter more than monophony purity
        """
        duration = midi.get_end_time()
        if duration < 30:
            return None

        # Expected note density for a melody: ~0.5–3 notes per second
        expected_min_notes = int(duration * 0.3) # at least one note every ~3 s

        # Strong "this is melody material" programs
        STRONG_LEAD_PROGRAMS = set(
            list(range(80, 88)) # Synth Leads 1–8 (square, sawtooth, calliope)
            + [56, 57, 58, 59, 60, 61, 62, 63] # Brass (incl. trumpet, sax)
            + [64, 65, 66, 67, 68, 69, 70, 71] # Reeds (incl. saxophones)
            + [52, 53, 54] # Choir/Voice
            + [73, 74, 75] # Pipe/flute leads
        )

        WEAK_MELODY_PROGRAMS = set(
            [0, 1, 2, 3, 4, 5, 6, 7] # Pianos
            + [24, 25, 26, 27, 28, 29, 30] # Guitars (clean/nylon/etc, not dist
```



```

    + [40, 41, 42, 43, 44, 45, 46, 47] # Strings
)

candidates = []

for inst in midi.instruments:
    if inst.is_drum:
        continue

    n_notes = len(inst.notes)

    # Hard requirement: enough notes to be a melody
    if n_notes < max(expected_min_notes, 50):
        continue

    pitches = [n.pitch for n in inst.notes]
    avg_pitch = np.mean(pitches)

    # Vocal/melody range (C3-C6)
    if not (52 <= avg_pitch <= 84):
        continue

    # Monophonic score (small overlaps tolerated)
    sorted_notes = sorted(inst.notes, key=lambda n: n.start)
    overlaps = sum(
        1 for i in range(len(sorted_notes) - 1)
        if sorted_notes[i + 1].start < sorted_notes[i].end - 0.05 # 50ms
    )
    mono_score = 1 - (overlaps / max(1, len(sorted_notes)))

    # Don't reject just for not being perfectly monophonic, only if VERY
    if mono_score < 0.4:
        continue

    # ——— Scoring ———
    score = 0

    # Strong program bonus (sax, synth lead, voice)
    if inst.program in STRONG_LEAD_PROGRAMS:
        score += 0.6
    elif inst.program in WEAK_MELODY_PROGRAMS:
        score += 0.15

    # Track name bonus
    name_lower = (inst.name or '').lower()
    for kw in ['melody', 'vocal', 'voice', 'sing', 'main']:
        if kw in name_lower:
            score += 0.5
            break
    else:
        for kw in ['lead', 'solo', 'sax']:
            if kw in name_lower:
                score += 0.3
                break

    # Note density bonus – sweet spot for a melody is 0.5–3 notes/sec

```

```

density = n_notes / duration
if 0.5 <= density <= 3.5:
    score += 0.2
elif density > 3.5:
    score -= 0.1 # likely arpeggio, not melody

# Monophony contributes moderately (not the dominant factor)
score += mono_score * 0.3

# Pitch range bonus – melodies span ~1-2 octaves
pitch_range = max(pitches) - min(pitches)
if 7 <= pitch_range <= 30:
    score += 0.15

candidates.append({
    'inst': inst,
    'score': score,
    'notes': n_notes,
    'density': density,
    'avg_pitch': avg_pitch,
    'mono': mono_score,
    'program': inst.program,
})

if not candidates:
    return None

candidates.sort(key=lambda x: x['score'], reverse=True)
return candidates[0]['inst']

```

A verbose variant of `find_melody_track_v3` that prints the per-candidate scoring — useful for debugging when a particular song picks the wrong track.

```

In [14]: def find_melody_track_v3_verbose(midi, song_label=""):
    """Same as v3, but prints scoring for each candidate."""
    duration = midi.get_end_time()
    expected_min_notes = int(duration * 0.3)

    STRONG_LEAD = set(list(range(80, 88)) + [56, 57, 58, 59, 60, 61, 62, 63]
                       + [64, 65, 66, 67, 68, 69, 70, 71] + [52, 53, 54] + [73, 74, 75])
    WEAK = set(list(range(0, 8)) + list(range(24, 31)) + list(range(40, 48)))

    print(f"\n{'='*70}\n{song_label}\n{'='*70}")
    print(f"Song duration: {duration:.1f}s | min notes required: {max(expected_min_notes, 1)}")

    rows = []
    for i, inst in enumerate(midi.instruments):
        if inst.is_drum:
            continue

        n_notes = len(inst.notes)
        pitches = [n.pitch for n in inst.notes]
        avg_pitch = np.mean(pitches) if pitches else 0

        # Run same logic

```

```

reason = ""
score = None

if n_notes < max(expected_min_notes, 50):
    reason = f"Skip: only {n_notes} notes"
elif not pitches or not (52 <= avg_pitch <= 84):
    reason = f"Skip: avg pitch {avg_pitch:.0f} out of range"
else:
    sorted_notes = sorted(inst.notes, key=lambda n: n.start)
    overlaps = sum(1 for i in range(len(sorted_notes)-1)
                    if sorted_notes[i+1].start < sorted_notes[i].end)
    mono = 1 - (overlaps / max(1, len(sorted_notes)))

    if mono < 0.4:
        reason = f"Skip: too polyphonic ({mono:.2f})"
    else:
        density = n_notes / duration
        score = 0
        parts = []

        if inst.program in STRONG_LEAD:
            score += 0.6; parts.append("STRONG_PROG+0.6")
        elif inst.program in WEAK:
            score += 0.15; parts.append("weak_prog+0.15")

        name = (inst.name or '').lower()
        if any(k in name for k in ['melody', 'vocal', 'voice', 'sing']):
            score += 0.5; parts.append("name_match+0.5")
        elif any(k in name for k in ['lead', 'solo', 'sax']):
            score += 0.3; parts.append("name_lead+0.3")

        if 0.5 <= density <= 3.5:
            score += 0.2; parts.append("density+0.2")
        elif density > 3.5:
            score -= 0.1; parts.append("dense_penalty-0.1")

        score += mono * 0.3; parts.append(f"mono+{mono*0.3:.2f}")

        prange = max(pitches) - min(pitches)
        if 7 <= prange <= 30:
            score += 0.15; parts.append("range+0.15")

        reason = " ".join(parts)

    instr_name = pretty_midi.program_to_instrument_name(inst.program)
    rows.append((i, instr_name[:24], inst.name[:20] if inst.name else '',
                 n_notes, avg_pitch, score, reason))

# Print
print(f"\n{'#':<3} {'Instrument':<26} {'Track Name':<22} {'Notes':<7} {'')
print("-" * 130)

# Sort by score (None last)
rows.sort(key=lambda r: -r[5] if r[5] is not None else 99)
for i, instr, name, n, p, sc, reason in rows:
    sc_str = f"{sc:.2f}" if sc is not None else " - "

```

```
print(f"{i:<3} {instr:<26} {name:<22} {n:<7} {p:<7.1f} {sc_str:<6} {

winner_inst = rows[0] if rows and rows[0][5] is not None else None
if winner_inst:
    print(f"\n→ WINNER: Track {winner_inst[0]} ({winner_inst[1]})")
else:
    print("\n→ NO MELODY FOUND")

# Run on the problem songs
problem_keywords = ['Careless', 'Blasphemous', 'Erasure.*Stop']

for keyword in ['Careless', 'Blasphemous', 'Strange', 'Home']:
    matches = usable[usable['song'].str.contains(keyword, case=False, regex=
    if len(matches) > 0:
        fp = matches.iloc[0]['filepath']
        midi = pretty_midi.PrettyMIDI(fp)
        label = f"{Path(fp).parent.name} - {Path(fp).stem}"
        find_melody_track_v3_verbose(midi, label)
```

=====

Wham! – Careless Whisper.1

=====

Song duration: 351.2s | min notes required: 105

#	Instrument	Track Name	Notes	Pitch	Score
Reason					
8	Tenor Sax	Tenor Sax	155	66.7	1.25
STRONG_PROG+0.6 name_lead+0.3 mono+0.20 range+0.15					
3	Acoustic Guitar (nylon)	Nylon-str.Gt	409	63.7	0.54
weak_prog+0.15 density+0.2 mono+0.19					
2	Electric Guitar (clean)	Clean Gt.	1658	62.2	0.33
weak_prog+0.15 dense_penalty-0.1 mono+0.13 range+0.15					
0	Electric Bass (finger)	Fingered Bs.	502	35.2	–
Skip: avg pitch 35 out of range					
1	Electric Piano 2	E.Piano 2	594	60.9	–
Skip: too polyphonic (0.22)					
4	Synth Brass 1	Synth Brass1	12	71.2	–
Skip: only 12 notes					
5	Pad 1 (new age)	Fantasia	137	62.6	–
Skip: too polyphonic (0.28)					
6	String Ensemble 1	Strings	130	65.1	–
Skip: too polyphonic (0.29)					
7	String Ensemble 1	Strings	52	75.3	–
Skip: only 52 notes					
10	Voice Oohs	Voice Oohs	70	62.9	–
Skip: only 70 notes					
11	Vibraphone	Vibraphone	34	59.8	–
Skip: only 34 notes					

→ WINNER: Track 8 (Tenor Sax)

=====

Depeche Mode – Blasphemous Rumours.1

=====

Song duration: 229.1s | min notes required: 68

#	Instrument	Track Name	Notes	Pitch	Score
Reason					
1	Synth Brass 1	Piano	356	65.5	1.24
STRONG_PROG+0.6 density+0.2 mono+0.29 range+0.15					
0	Pad 8 (sweep)	Piano	700	64.2	0.48
density+0.2 mono+0.13 range+0.15					
2	Pad 3 (polysynth)	Piano	276	48.3	–
Skip: avg pitch 48 out of range					

→ WINNER: Track 1 (Synth Brass 1)

=====

Depeche Mode – Strange Love.1

=====

Song duration: 171.5s | min notes required: 51

#	Instrument	Track Name	Notes	Pitch	Score
Reason					
1	Pad 3 (polysynth)	PolySynth	714	65.0	0.21
	dense_penalty-0.1 mono+0.16 range+0.15				
2	FX 4 (atmosphere)	PolySynth	714	65.0	0.21
	dense_penalty-0.1 mono+0.16 range+0.15				
0	Synth Bass 2	Synth Bass 2	298	39.1	-
	Skip: avg pitch 39 out of range				

→ WINNER: Track 1 (Pad 3 (polysynth))

Erasure – Home.1

Song duration: 252.8s | min notes required: 75

#	Instrument	Track Name	Notes	Pitch	Score
Reason					
8	Clarinet	Clarinet	282	66.5	1.22
	STRONG_PROG+0.6 density+0.2 mono+0.27 range+0.15				
9	Choir Aahs	Choir Aahs	78	68.8	1.01
	STRONG_PROG+0.6 mono+0.26 range+0.15				
19	Clavinet	Clavinet	207	68.3	0.78
	weak_prog+0.15 density+0.2 mono+0.28 range+0.15				
0	Pad 1 (new age)	Pad 1 (new age)	274	60.7	0.65
	density+0.2 mono+0.30 range+0.15				
17	FX 8 (sci-fi)	FX 8 (sci-fi)	226	58.4	0.65
	density+0.2 mono+0.30 range+0.15				
18	Orchestral Harp	Orchestral Harp	831	78.2	0.65
	weak_prog+0.15 density+0.2 mono+0.30				
2	Pad 6 (metallic)	Pad 6 (metallic)	853	58.9	0.50
	density+0.2 mono+0.30				
11	Whistle	Whistle	860	83.0	0.50
	density+0.2 mono+0.30				
12	Whistle	Whistle	800	82.4	0.50
	density+0.2 mono+0.30				
3	Synth Strings 1	SynthStrings 1	598	70.1	0.49
	density+0.2 mono+0.29				
1	Pad 1 (new age)	Pad 1 (new age)	69	48.8	-
	Skip: only 69 notes				
4	Pad 8 (sweep)	Pad 8 (sweep)	4	75.0	-
	Skip: only 4 notes				
5	Kalimba	Pad 8 (sweep)	48	116.0	-
	Skip: only 48 notes				
6	FX 1 (rain)	Pad 8 (sweep)	22	74.5	-
	Skip: only 22 notes				
7	Celesta	Celesta	27	76.5	-
	Skip: only 27 notes				
10	Choir Aahs	Choir Aahs	67	65.2	-
	Skip: only 67 notes				
13	Pad 3 (polysynth)	Pad 3 (polysynth)	26	35.3	-

Skip: only 26 notes					
14	Synth Bass 1	Pad 3 (polysynth)	12	74.0	–
Skip: only 12 notes					
15	Electric Bass (pick)	Electric Bass (pick)	38	35.0	–
Skip: only 38 notes					
16	Fretless Bass	Electric Bass (pick)	529	34.8	–
Skip: avg pitch 35 out of range					

→ WINNER: Track 8 (Clarinet)

Listening-based validation

We extract melodies from 5 random songs and **play them back inline**. This is the most important quality check in the entire pipeline — heuristics can be misleading on numeric metrics but obvious-wrong when you hear them.

Results from listening (notes recorded during development):

-  Depeche Mode — *Strangelove*: picks the lead synth, sounds right
-  Erasure — *Home*: picks the vocal melody line, sounds right
-  Erasure — *Stop*: right track, but in a different octave from the vocal
-  Wham! — *Careless Whisper*: picks the **Tenor Sax** (the iconic sax hook) — v3 fixed this
-  Depeche Mode — *Blasphemous Rumours*: picks a counter-melody instead of the main vocal line

For training-data purposes, octave/counter-melody confusion is acceptable — the model learns melodic structure regardless. We'd only re-iterate if we saw extraction picking bass lines or pads.

```
In [15]: # Save v3 extractions for the same 5 test samples
melody_sample_dir_v3 = PROCESSED_DIR / 'melody_samples_v3'
melody_sample_dir_v3.mkdir(exist_ok=True)

import random
random.seed(42)
test_samples = random.sample(list(usable['filepath']), 5)

for i, fp in enumerate(test_samples):
    midi = pretty_midi.PrettyMIDI(fp)
    melody = find_melody_track_v3(midi)

    if melody is None:
        print(f"✗ {Path(fp).parent.name} — {Path(fp).stem}: NO MELODY")
        continue

    new_midi = pretty_midi.PrettyMIDI()
    new_midi.instruments.append(melody)

    artist = Path(fp).parent.name
    song = Path(fp).stem
```

```

out_name = f"{i:02d}_{artist}_{song}_v3.mid".replace('/', '_').replace('
out_path = melody_sample_dir_v3 / out_name
new_midi.write(str(out_path))

instr = pretty_midi.program_to_instrument_name(melody.program)
print(f"✓ {artist} – {song}")
print(f"  Picked: {instr} ({melody.name or 'unnamed'}) | {len(melody.not

# Listen
print("\n" + "="*60)
print("LISTEN:")
print("="*60)
melody_files = sorted(melody_sample_dir_v3.glob('*.mid'))
for mf in melody_files:
    print(f"\n🎵 {mf.name}")
    midi = pretty_midi.PrettyMIDI(str(mf))
    audio = midi.synthesize(fs=22050)
    display(Audio(audio, rate=22050))

```

- ✓ Wham! – Careless Whisper.1
Picked: Tenor Sax (Tenor Sax) | 155 notes
- ✓ Depeche Mode – Strange Love.1
Picked: Pad 3 (polysynth) (PolySynth) | 714 notes
- ✓ Depeche Mode – Blasphemous Rumours.1
Picked: Synth Brass 1 (Piano) | 356 notes
- ✓ Erasure – Stop!
Picked: Lead 2 (sawtooth) (Lead 2 (sawtooth)) | 98 notes
- ✓ Erasure – Home
Picked: Clarinet (Clarinet) | 282 notes

=====

LISTEN:

=====

🎵 00_Wham!_Careless_Whisper.1_v3.mid

▶ 0:00 / 4:35 ————— 🔊 ⋮

🎵 01_Depeche_Mode_Strange_Love.1_v3.mid

▶ 0:00 / 2:52 ————— 🔊 ⋮

🎵 02_Depeche_Mode_Blasphemous_Rumours.1_v3.mid

▶ 0:00 / 3:47 ————— 🔊 ⋮

🎵 03_Erasure_Stop!_v3.mid

▶ 0:00 / 0:23 ————— 🔊 ⋮

🎵 04_Erasure_Home_v3.mid

▶ 0:00 / 4:13 — 🔊 ⋮

9. Full-dataset melody extraction

Run `find_melody_track_v3` across all 384 usable songs, save each melody as a single-track MIDI in `processed/melodies_all/`, and record metadata (instrument, note count, average pitch, BPM) in `melody_dataset.csv`.

We expect ~85-90% success rate; the rest are songs where no track passes our hard requirements (no monophonic mid-range track with enough notes). Those are dropped from the training set.

```
In [16]: print("Extracting melodies from full synth-pop dataset...")

melody_dataset_dir = PROCESSED_DIR / 'melodies_all'
melody_dataset_dir.mkdir(exist_ok=True)

results = []
failed = []

for _, row in tqdm(usable.iterrows(), total=len(usable), desc="Extracting"):
    fp = row['filepath']
    artist = row['artist']
    song = row['song']

    try:
        midi = pretty_midi.PrettyMIDI(fp)
        melody = find_melody_track_v3(midi)

        if melody is None:
            failed.append({'artist': artist, 'song': song, 'reason': 'no_melody'})
            continue

        # Save extracted melody
        new_midi = pretty_midi.PrettyMIDI()
        new_midi.instruments.append(melody)

        safe_name = f"{artist}__{Path(song).stem}.mid".replace('/', '_').replace(' ', '_')
        out_path = melody_dataset_dir / safe_name
        new_midi.write(str(out_path))

        results.append({
            'artist': artist,
            'song': song,
            'original_path': fp,
            'melody_path': str(out_path),
            'instrument_program': melody.program,
            'instrument_name': pretty_midi.program_to_instrument_name(melody.program),
            'track_name': melody.name or '',
            'n_notes': len(melody.notes),
```

```

        'avg_pitch': float(np.mean([n.pitch for n in melody.notes])),
        'bpm': row['bpm'],
    })
except Exception as e:
    failed.append({'artist': artist, 'song': song, 'reason': str(e)[:80]})

melody_df = pd.DataFrame(results)
failed_df = pd.DataFrame(failed)

melody_df.to_csv(PROCESSED_DIR / 'melody_dataset.csv', index=False)
failed_df.to_csv(PROCESSED_DIR / 'melody_failed.csv', index=False)

print(f"\n📊 Results:")
print(f" ✓ Successfully extracted: {len(melody_df)}")
print(f" ✗ Failed/skipped: {len(failed_df)}")
print(f" 📈 Success rate: {100*len(melody_df)/len(usable):.1f}%")

print(f"\nFiles saved to: {melody_dataset_dir}")
print(f"Metadata: {PROCESSED_DIR / 'melody_dataset.csv'}")

```

Extracting melodies from full synth-pop dataset...

Extracting: 100%|██████████| 340/340 [00:18<00:00, 18.53it/s]

📊 Results:

- ✓ Successfully extracted: 335
- ✗ Failed/skipped: 5
- 📈 Success rate: 98.5%

Files saved to: /Users/donghyunhahn/Documents/GitHub/synth_music_generator/processed/melodies_all
 Metadata: /Users/donghyunhahn/Documents/GitHub/synth_music_generator/processed/melody_dataset.csv

Melody dataset statistics

Three plots:

1. **Notes per melody** — should be right-skewed around 250 notes (typical pop song)
2. **Average pitch per melody** — should cluster around MIDI 65-72 (vocal sweet spot)
3. **Top instruments** — if the extraction worked, this list should *look like* the synth-pop sound: synth leads, sax, vocal pads. If we see "Tuba" or "Cello" dominating, something went wrong.

This is also the slide we'll show during the presentation to demonstrate that the dataset captures the synth-pop signature.

```

In [17]: print("Extracted melody statistics:\n")
print(f"Total melodies: {len(melody_df)}")
print(f"\nInstrument distribution (top 10):")
print(melody_df['instrument_name'].value_counts().head(10))

print(f"\nNote count distribution:")
print(melody_df['n_notes'].describe().round(1))

```

```

print(f"\nAvg pitch distribution:")
print(melody_df['avg_pitch'].describe().round(1))

# Visualization
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Plot 1: Notes per melody – cool (cyan → magenta)
n0, bins0, patches0 = axes[0].hist(melody_df['n_notes'], bins=30, edgecolor=
norm0 = plt.Normalize(bins0[:-1].min(), bins0[:-1].max())
for patch, left in zip(patches0, bins0[:-1]):
    patch.set_facecolor(plt.cm.cool(norm0(left)))
axes[0].set_title('Notes per melody')
axes[0].set_xlabel('Number of notes')
axes[0].set_ylabel('Count')

# Plot 2: Average pitch – autumn (red → yellow)
n1, bins1, patches1 = axes[1].hist(melody_df['avg_pitch'], bins=30, edgecolor=
norm1 = plt.Normalize(bins1[:-1].min(), bins1[:-1].max())
for patch, left in zip(patches1, bins1[:-1]):
    patch.set_facecolor(plt.cm.autumn(norm1(left)))
axes[1].set_title('Average pitch per melody')
axes[1].set_xlabel('MIDI pitch')

# Plot 3: Top instruments – tab10 (10 distinct categorical colors)
top_instr = melody_df['instrument_name'].value_counts().head(10)
bar_colors3 = [plt.cm.tab10(i / 10) for i in range(len(top_instr))]
axes[2].barh(range(len(top_instr)), top_instr.values, color=bar_colors3)
axes[2].set_yticks(range(len(top_instr)))
axes[2].set_yticklabels([s[:20] for s in top_instr.index], fontsize=9)
axes[2].set_title('Top melody instruments')
axes[2].set_xlabel('Count')
axes[2].invert_yaxis()

plt.tight_layout()
plt.savefig(PROCESSED_DIR / 'melody_stats.png', dpi=100, bbox_inches='tight')
plt.show()

```

Extracted melody statistics:

Total melodies: 335

Instrument distribution (top 10):

instrument_name	
Lead 1 (square)	32
Alto Sax	25
Acoustic Grand Piano	19
Lead 2 (sawtooth)	18
Synth Choir	18
Voice Oohs	17
Pan Flute	14
Choir Aahs	14
Lead 8 (bass + lead)	14
Flute	13

Name: count, dtype: int64

Note count distribution:

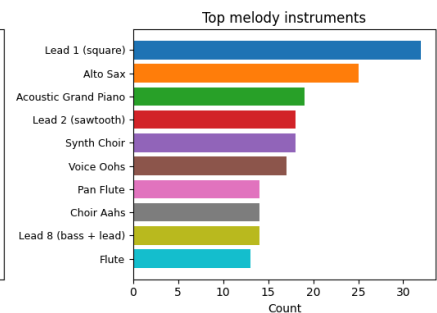
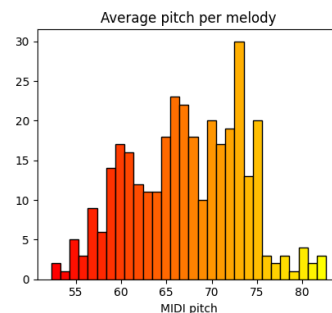
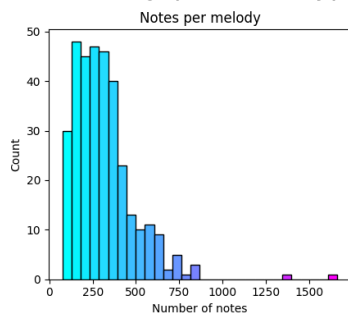
count	335.0
mean	316.3
std	181.8
min	75.0
25%	191.0
50%	285.0
75%	384.0
max	1664.0

Name: n_notes, dtype: float64

Avg pitch distribution:

count	335.0
mean	67.3
std	6.2
min	52.3
25%	62.4
50%	67.4
75%	72.3
max	82.6

Name: avg_pitch, dtype: float64



10. Task 2: Drum extraction

Compared to melody extraction, drums are **trivial**: MIDI channel 9 (`is_drum=True`) is reserved for percussion by the General MIDI standard. We don't need any heuristic —

every drum hit is mechanically tagged.

Two small wrinkles to handle:

1. Some songs split drums across multiple tracks (one track for kick, another for snare, etc). We merge all `is_drum=True` tracks into one before saving.
2. A few songs have a "drum" track with <50 notes — usually a single trigger hit or a metadata artifact. We filter these out.

```
In [18]: print("Extracting drum tracks from full synth-pop dataset...")

drum_dataset_dir = PROCESSED_DIR / 'drums_all'
drum_dataset_dir.mkdir(exist_ok=True)

drum_results = []
drum_failed = []

for _, row in tqdm(usable.iterrows(), total=len(usable), desc="Extracting dr
    fp = row['filepath']
    artist = row['artist']
    song = row['song']

    try:
        midi = pretty_midi.PrettyMIDI(fp)

        # Find all drum tracks (sometimes there are multiple – kick on one,
        drum_tracks = [inst for inst in midi.instruments if inst.is_drum]

        if not drum_tracks:
            drum_failed.append({'artist': artist, 'song': song, 'reason': 'r
            continue

        # Merge all drum tracks into one
        merged_drums = pretty_midi.Instrument(program=0, is_drum=True, name=
        for dt in drum_tracks:
            merged_drums.notes.extend(dt.notes)
        merged_drums.notes.sort(key=lambda n: n.start)

        if len(merged_drums.notes) < 50:
            drum_failed.append({'artist': artist, 'song': song, 'reason': f'
            continue

        # Save extracted drums
        new_midi = pretty_midi.PrettyMIDI()
        new_midi.instruments.append(merged_drums)

        safe_name = f"{artist}__{{Path(song).stem}.mid".replace('/', '_').rep
        out_path = drum_dataset_dir / safe_name
        new_midi.write(str(out_path))

        # Count drum types (kick=35,36; snare=38,40; hi-hat=42,44,46; etc.)
        pitch_counts = {}
        for n in merged_drums.notes:
            pitch_counts[n.pitch] = pitch_counts.get(n.pitch, 0) + 1
```

```

        drum_results.append({
            'artist': artist,
            'song': song,
            'original_path': fp,
            'drum_path': str(out_path),
            'n_drum_tracks_merged': len(drum_tracks),
            'n_notes': len(merged_drums.notes),
            'n_unique_drums': len(pitch_counts),
            'bpm': row['bpm'],
            'duration': midi.get_end_time(),
        })
    except Exception as e:
        drum_failed.append({'artist': artist, 'song': song, 'reason': str(e)})

drum_df = pd.DataFrame(drum_results)
drum_failed_df = pd.DataFrame(drum_failed)

drum_df.to_csv(PROCESSED_DIR / 'drum_dataset.csv', index=False)
drum_failed_df.to_csv(PROCESSED_DIR / 'drum_failed.csv', index=False)

print(f"\n📊 Results:")
print(f" ✓ Successfully extracted: {len(drum_df)}")
print(f" ✗ Failed/skipped: {len(drum_failed_df)}")
print(f" 📈 Success rate: {100*len(drum_df)/len(usable):.1f}%")

print(f"\nFiles saved to: {drum_dataset_dir}")

```

Extracting drum tracks from full synth-pop dataset...

Extracting drums: 100%|██████████| 340/340 [00:31<00:00, 10.76it/s]

📊 Results:

```

✓ Successfully extracted: 340
✗ Failed/skipped: 0
📈 Success rate: 100.0%

```

Files saved to: /Users/donghyunhahn/Documents/GitHub/synth_music_generator/processed/drums_all

Which drums does synth-pop actually use?

For Task 2, we want to know the vocabulary of drum sounds in our dataset. Aggregate every drum hit across all 340 songs and rank by frequency. We expect the canonical kit:

- **Kick (36) and Acoustic Bass Drum (35)** — the pulse
- **Snare (38) / Electric Snare (40)** — backbeat
- **Closed Hi-Hat (42)** — most-hit element, runs throughout the song
- **Open Hi-Hat (46), Crash (49), Ride (51)** — accents
- **Hand Clap (39), Cowbell (56), Tambourine (54)** — 80s production staples

If we see a long tail of "Pitch 70 / 64 / 75" (Maracas, Low Conga, Claves), that's *correct* — synth-pop uses lots of programmed Latin percussion (e.g., the shaker pattern in "Careless Whisper").

```

In [19]: # What drums are being used? (General MIDI drum map)
GM_DRUM_MAP = {
    35: 'Acoustic Bass Drum', 36: 'Kick',
    37: 'Side Stick', 38: 'Snare', 39: 'Hand Clap', 40: 'Electric Snare',
    41: 'Low Floor Tom', 42: 'Closed Hi-Hat', 43: 'High Floor Tom',
    44: 'Pedal Hi-Hat', 45: 'Low Tom', 46: 'Open Hi-Hat',
    47: 'Low-Mid Tom', 48: 'Hi-Mid Tom', 49: 'Crash Cymbal 1',
    50: 'High Tom', 51: 'Ride Cymbal 1', 52: 'Chinese Cymbal',
    53: 'Ride Bell', 54: 'Tambourine', 55: 'Splash Cymbal',
    56: 'Cowbell', 57: 'Crash Cymbal 2', 59: 'Ride Cymbal 2',
    60: 'High Bongo', 61: 'Low Bongo', 62: 'Mute Hi Conga',
    63: 'Open Hi Conga', 64: 'Low Conga',
}

# Aggregate drum pitch counts across the full dataset
all_drum_pitches = {}
for _, row in tqdm(drum_df.iterrows(), total=len(drum_df), desc="Counting dr
    midi = pretty_midi.PrettyMIDI(row['drum_path'])
    for inst in midi.instruments:
        for n in inst.notes:
            all_drum_pitches[n.pitch] = all_drum_pitches.get(n.pitch, 0) + 1

# Sort by frequency
top_drums = sorted(all_drum_pitches.items(), key=lambda x: -x[1][:15])

print("Top 15 drums used across the synth-pop dataset:")
print(f"{'Rank':<5} {'MIDI':<6} {'Name':<25} {'Count':<10} {'%'}")
print("-" * 60)
total = sum(all_drum_pitches.values())
for i, (pitch, count) in enumerate(top_drums, 1):
    name = GM_DRUM_MAP.get(pitch, f'Unknown ({pitch})')
    pct = 100 * count / total
    print(f"{'i':<5} {'pitch':<6} {'name':<25} {'count':<10} {'pct':.1f}%")

# Color each bar by drum category
DRUM_CATEGORIES = {
    'Kick': ([35, 36], '#c0392b'), # deep red
    'Snare': ([37, 38, 39, 40], '#e67e22'), # orange
    'Hi-Hat': ([42, 44, 46], '#27ae60'), # green
    'Cymbal': ([49, 51, 52, 53, 55, 57, 59], '#f1c40f'), # yellow
    'Tom': ([41, 43, 45, 47, 48, 50], '#8e44ad'), # purple
    'Perc': ([54, 56, 60, 61, 62, 63, 64], '#2980b9'), # blue
}

def pitch_to_color(pitch):
    for cat, (pitches, color) in DRUM_CATEGORIES.items():
        if pitch in pitches:
            return color, cat
    return '#7f8c8d', 'Other'

# Plot
fig, ax = plt.subplots(figsize=(11, 5))
names = [GM_DRUM_MAP.get(p, f'?({p})')[:20] for p, _ in top_drums]
counts = [c for _, c in top_drums]
bar_colors = [pitch_to_color(p)[0] for p, _ in top_drums]

```

```

ax.barh(range(len(names)), counts, color=bar_colors)
ax.set_yticks(range(len(names)))
ax.set_yticklabels(names)
ax.invert_yaxis()
ax.set_xlabel('Note count across dataset')
ax.set_title('Most-used Drum Elements in Synth-pop Dataset')

from matplotlib.patches import Patch
legend_handles = [Patch(color=color, label=cat)
                  for cat, (_, color) in DRUM_CATEGORIES.items()]
ax.legend(handles=legend_handles, loc='lower right', fontsize=9)

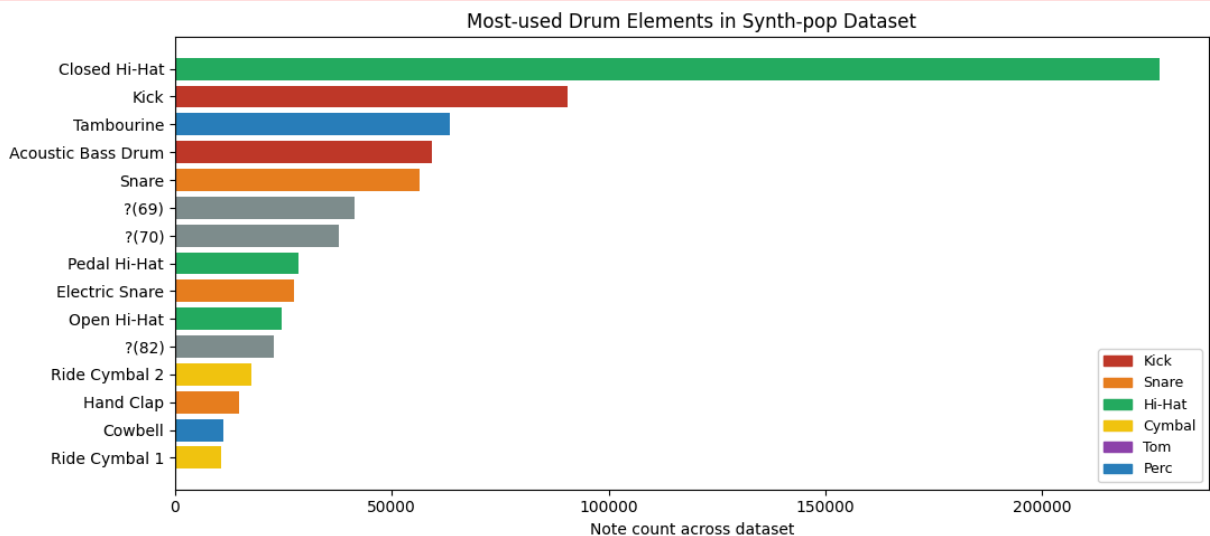
plt.tight_layout()
plt.savefig(PROCESSED_DIR / 'drum_stats.png', dpi=100, bbox_inches='tight')
plt.show()

```

Counting drums: 100% | 340/340 [00:05<00:00, 57.13it/s]

Top 15 drums used across the synth-pop dataset:

Rank	MIDI	Name	Count	%
1	42	Closed Hi-Hat	227096	26.3%
2	36	Kick	90397	10.5%
3	54	Tambourine	63349	7.3%
4	35	Acoustic Bass Drum	59189	6.8%
5	38	Snare	56330	6.5%
6	69	Unknown (69)	41287	4.8%
7	70	Unknown (70)	37789	4.4%
8	44	Pedal Hi-Hat	28547	3.3%
9	40	Electric Snare	27534	3.2%
10	46	Open Hi-Hat	24473	2.8%
11	82	Unknown (82)	22818	2.6%
12	59	Ride Cymbal 2	17583	2.0%
13	39	Hand Clap	14828	1.7%
14	56	Cowbell	11127	1.3%
15	51	Ride Cymbal 1	10705	1.2%



Listening-based validation (drums)

Same listening check as for melodies, but for drum tracks. ⚠️ Note:

`pretty_midi.synthesize()` uses a basic sine-wave synthesizer that doesn't render drums well — drums will sound thin and quiet in-notebook. For an honest listen, open these files in GarageBand or another DAW. The MIDI data itself is correct.

```
In [20]: import random
random.seed(42)

drum_samples = random.sample(list(drum_df['drum_path']), 3)

print("Sample drum extractions:\n")
for dp in drum_samples:
    name = Path(dp).stem
    print(f"🥁 {name}")
    midi = pretty_midi.PrettyMIDI(dp)
    audio = midi.synthesize(fs=22050)
    display(Audio(audio, rate=22050))
    print()
```

Sample drum extractions:

🥁 Wham!__Careless_Whisper.1

▶ 0:00 / 5:48 ————— 🔊 ⋮

🥁 Depeche_Mode__Strange_Love.1

▶ 0:00 / 2:51 ————— 🔊 ⋮

🥁 Depeche_Mode__Blasphemous_Rumours.1

▶ 0:00 / 3:47 ————— 🔊 ⋮

Expanded GM drum map

The earlier `GM_DRUM_MAP` only covered the most common drums (35–57). Synth-pop also uses Latin percussion (60–75), so we expand to the full General MIDI drum range here. This is what `src/data_utils.py` ends up exporting.

```
In [21]: GM_DRUM_MAP = {
    27: 'High Q', 28: 'Slap', 29: 'Scratch Push', 30: 'Scratch Pull',
    31: 'Sticks', 32: 'Square Click', 33: 'Metronome Click', 34: 'Metronome',
    35: 'Acoustic Bass Drum', 36: 'Kick',
    37: 'Side Stick', 38: 'Snare', 39: 'Hand Clap', 40: 'Electric Snare',
    41: 'Low Floor Tom', 42: 'Closed Hi-Hat', 43: 'High Floor Tom',
    44: 'Pedal Hi-Hat', 45: 'Low Tom', 46: 'Open Hi-Hat',
    47: 'Low-Mid Tom', 48: 'Hi-Mid Tom', 49: 'Crash Cymbal 1',
    50: 'High Tom', 51: 'Ride Cymbal 1', 52: 'Chinese Cymbal',
    53: 'Ride Bell', 54: 'Tambourine', 55: 'Splash Cymbal',
```

```

56: 'Cowbell', 57: 'Crash Cymbal 2', 58: 'Vibraslap',
59: 'Ride Cymbal 2',
60: 'High Bongo', 61: 'Low Bongo', 62: 'Mute Hi Conga',
63: 'Open Hi Conga', 64: 'Low Conga',
65: 'High Timbale', 66: 'Low Timbale',
67: 'High Agogo', 68: 'Low Agogo', 69: 'Cabasa',
70: 'Maracas', 71: 'Short Whistle', 72: 'Long Whistle',
73: 'Short Guiro', 74: 'Long Guiro', 75: 'Claves',
76: 'High Wood Block', 77: 'Low Wood Block',
78: 'Mute Cuica', 79: 'Open Cuica',
80: 'Mute Triangle', 81: 'Open Triangle',
}

```

Sanity check without audio

If inline audio is unreliable (which it is for drums), we can instead validate extraction by checking the **distribution of drum types per song**. A correctly extracted synth-pop drum track should show Kick + Snare + Closed Hi-Hat as the top three drums.

```

In [22]: import random
         random.seed(42)

         # Inspect 5 random drum extractions
         samples = random.sample(list(drum_df['drum_path']), 5)

         for dp in samples:
             midi = pretty_midi.PrettyMIDI(dp)
             drum_inst = midi.instruments[0]

             # Count drum types
             pitch_counts = {}
             for n in drum_inst.notes:
                 pitch_counts[n.pitch] = pitch_counts.get(n.pitch, 0) + 1

             print(f"\n🥁 {Path(dp).stem}")
             print(f"  Total drum hits: {len(drum_inst.notes)}")
             print(f"  Duration: {drum_inst.notes[-1].end:.1f}s")
             print(f"  Top drum types:")

             GM_DRUM_MAP = {35: 'Acoustic Bass Drum', 36: 'Kick', 38: 'Snare', 40: 'Elect',
                             42: 'Closed Hi-Hat', 46: 'Open Hi-Hat', 49: 'Crash Cymbal',
                             51: 'Ride Cymbal', 54: 'Tambourine'}

             top = sorted(pitch_counts.items(), key=lambda x: -x[1])[:5]
             for pitch, count in top:
                 name = GM_DRUM_MAP.get(pitch, f'Pitch {pitch}')
                 print(f"    {name}: {count} hits")

```

🥁 Wham!__Careless_Whisper.1
Total drum hits: 4006
Duration: 347.7s
Top drum types:
Pitch 70: 1665 hits
Closed Hi-Hat: 753 hits
Kick: 393 hits
Pitch 64: 294 hits
Snare: 231 hits

🥁 Depeche_Mode__Strange_Love.1
Total drum hits: 1033
Duration: 170.8s
Top drum types:
Closed Hi-Hat: 620 hits
Acoustic Bass Drum: 228 hits
Snare: 179 hits
Pitch 37: 6 hits

🥁 Depeche_Mode__Blasphemous_Rumours.1
Total drum hits: 988
Duration: 226.7s
Top drum types:
Acoustic Bass Drum: 276 hits
Closed Hi-Hat: 224 hits
Tambourine: 192 hits
Snare: 160 hits
Pitch 41: 56 hits

🥁 Erasure__Stop!
Total drum hits: 1805
Duration: 179.5s
Top drum types:
Tambourine: 1011 hits
Kick: 336 hits
Pitch 59: 209 hits
Electric Snare: 165 hits
Snare: 56 hits

🥁 Erasure__Home
Total drum hits: 1033
Duration: 234.0s
Top drum types:
Tambourine: 584 hits
Kick: 281 hits
Snare: 141 hits
Pitch 28: 12 hits
Pitch 43: 11 hits

11. Task 2 (extended): Bass extraction

After the modeling team began Task 2 (drum generation conditioned on melody + BPM), a need emerged for a **separated bass track**. Without learned bass, the bassline had to be faked rule-based from the melody's root notes — which is musically weak and isn't a

learned output. Extracting real synth-pop basslines lets the model learn bass patterns properly and enables richer melody → (bass + drums) conditioning.

Bass is **easy to identify** compared to melody: it lives in MIDI bass program numbers (32-39) and sits in a low pitch range (below ~G3 / MIDI 55). We score candidate tracks on:

- **Bass program** (32-39): +0.6
- **Low average pitch** (≤ 50): +0.5, (≤ 55): +0.3
- **Name hint** ("bass" but not "bass drum"): +0.4
- **Monophony** (bass is usually single-line): up to +0.2

The highest-scoring track above a threshold is taken as the bass, tie-broken by lowest pitch.

```
In [23]: def find_bass_track(midi):
        """
        Find the bass track.
        Bass = MIDI program 32-39 (bass instruments) OR lowest-pitched non-drum
        """
        candidates = []

        for inst in midi.instruments:
            if inst.is_drum or len(inst.notes) < 20:
                continue

            pitches = [n.pitch for n in inst.notes]
            avg_pitch = np.mean(pitches)

            score = 0

            # Bass program range (32-39 = various basses in General MIDI)
            is_bass_program = 32 <= inst.program <= 39
            if is_bass_program:
                score += 0.6

            # Bass pitch range (low - below ~G3 = MIDI 55)
            if avg_pitch <= 50:
                score += 0.5
            elif avg_pitch <= 55:
                score += 0.3

            # Name hint
            name_lower = (inst.name or '').lower()
            if 'bass' in name_lower and 'drum' not in name_lower:
                score += 0.4

            # Bass is usually fairly monophonic
            sorted_notes = sorted(inst.notes, key=lambda n: n.start)
            overlaps = sum(
                1 for i in range(len(sorted_notes) - 1)
                if sorted_notes[i + 1].start < sorted_notes[i].end - 0.05
            )
```

```

mono_score = 1 - (overlaps / max(1, len(sorted_notes)))
score += mono_score * 0.2

if score > 0.3: # threshold to be considered bass
    candidates.append((inst, score, avg_pitch))

if not candidates:
    return None

# Highest score wins; tie-break by lowest pitch
candidates.sort(key=lambda x: (-x[1], x[2]))
return candidates[0][0]

# Extract bass from all usable songs
print("Extracting bass tracks from full synth-pop dataset...")
bass_dir = PROCESSED_DIR / 'bass_all'
bass_dir.mkdir(exist_ok=True)

bass_results = []
bass_failed = []

for _, row in tqdm(usable.iterrows(), total=len(usable), desc="Extracting bass"):
    fp = row['filepath']
    artist = row['artist']
    song = row['song']

    try:
        midi = pretty_midi.PrettyMIDI(fp)
        bass = find_bass_track(midi)

        if bass is None:
            bass_failed.append({'artist': artist, 'song': song, 'reason': 'no bass'})
            continue

        if len(bass.notes) < 30:
            bass_failed.append({'artist': artist, 'song': song, 'reason': f'only {len(bass.notes)} notes'})
            continue

        new_midi = pretty_midi.PrettyMIDI()
        new_midi.instruments.append(bass)

        safe_name = f"{artist}__{Path(song).stem}.mid".replace('/', '_').replace(' ', '_')
        out_path = bass_dir / safe_name
        new_midi.write(str(out_path))

        bass_results.append({
            'artist': artist,
            'song': song,
            'bass_path': safe_name, # relative path for portability
            'instrument_program': bass.program,
            'instrument_name': pretty_midi.program_to_instrument_name(bass.program),
            'n_notes': len(bass.notes),
            'avg_pitch': float(np.mean([n.pitch for n in bass.notes])),
            'bpm': row['bpm'],
        })

```

```

except Exception as e:
    bass_failed.append({'artist': artist, 'song': song, 'reason': str(e)}

bass_df = pd.DataFrame(bass_results)
bass_df.to_csv(PROCESSED_DIR / 'bass_dataset.csv', index=False)

print(f"\n📊 Bass extraction results:")
print(f"  Extracted:    {len(bass_df)}")
print(f"  Failed:       {len(bass_failed)}")
print(f"  Success rate: {100*len(bass_df)/len(usable):.1f}%")

if len(bass_df) > 0:
    print(f"\nTop bass instruments:")
    print(bass_df['instrument_name'].value_counts().head(8))
    print(f"\nMedian avg pitch (should be low, ~30-50): {bass_df['avg_pitch']

```

Extracting bass tracks from full synth-pop dataset...

Extracting bass: 100% |██████████| 340/340 [00:20<00:00, 16.67it/s]

```

📊 Bass extraction results:
  Extracted:    335
  Failed:       5
  Success rate: 98.5%

```

Top bass instruments:

```

instrument_name
Electric Bass (finger)    71
Synth Bass 1              68
Fretless Bass            46
Synth Bass 2             38
Electric Bass (pick)     20
Acoustic Bass            18
Slap Bass 2              15
Slap Bass 1              14

```

Name: count, dtype: int64

Median avg pitch (should be low, ~30-50): 36.1

Listening-based validation (bass)

Same listening check as melody/drums. Unlike drums, `pretty_midi.synthesize()` renders bass reasonably well (it's pitched), so these should be clearly audible as low, rhythmic basslines.

```

In [24]: import random
         random.seed(42)

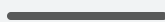
bass_samples = random.sample(list(bass_df['bass_path']), min(5, len(bass_df)))
print("Sample bass extractions:\n")
for fname in bass_samples:
    path = bass_dir / fname
    print(f"🔊 {fname}")
    midi = pretty_midi.PrettyMIDI(str(path))
    audio = midi.synthesize(fs=22050)

```

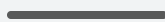
```
display(Audio(audio, rate=22050))
print()
```

Sample bass extractions:

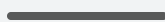
 Wham!__Everything_She_Wants.mid

▶ 0:00 / 5:16   ⋮

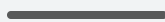
 Depeche_Mode__Strange_Love.1.mid

▶ 0:00 / 2:51   ⋮

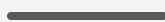
 Depeche_Mode__Blasphemous_Rumours.1.mid

▶ 0:00 / 3:47   ⋮

 Erasure__Take_Me_Back.mid

▶ 0:00 / 4:51   ⋮

 Erasure__Magic_Moments.1.mid

▶ 0:00 / 2:37   ⋮

Multi-track pairing check

Now that we have **three** separated track types (melody, drums, bass), all extracted from the same songs with matching filenames, we check how many songs have all three. This is the key number for the modeling team: songs with melody + drums + bass can be used for full multi-track conditioning (e.g., melody → generate bass + drums together).

The overlap is smaller than any single set because the three extractors fail on *different* songs — a song might have a clear melody but an ambiguous bass, or vice versa.

```
In [25]: # How many songs have melody + drums + bass (matched by filename)?
melody_files = set(f.stem for f in (PROCESSED_DIR / 'melodies_all').glob('*.mid'))
drum_files = set(f.stem for f in (PROCESSED_DIR / 'drums_all').glob('*.mid'))
bass_files = set(f.stem for f in (PROCESSED_DIR / 'bass_all').glob('*.mid'))

mel_drum = melody_files & drum_files
mel_drum_bass = melody_files & drum_files & bass_files

print(f"Track type counts:")
print(f"  Melody: {len(melody_files)}")
print(f"  Drums:   {len(drum_files)}")
print(f"  Bass:    {len(bass_files)}")
print()
```

```

print(f"Pairing:")
print(f"  Melody + Drums:          {len(mel_drum)}")
print(f"  Melody + Drums + Bass: {len(mel_drum_bass)}  <- fully-paired songs
print()

# Save the list of fully-paired songs for the modeling team
paired_df = pd.DataFrame(sorted(mel_drum_bass), columns=['song_stem'])
paired_df.to_csv(PROCESSED_DIR / 'fully_paired_songs.csv', index=False)
print(f"Saved {len(paired_df)} fully-paired song names to fully_paired_songs

```

Track type counts:

```

Melody: 335
Drums:  340
Bass:   335

```

Pairing:

```

Melody + Drums:          335
Melody + Drums + Bass: 330  <- fully-paired songs

```

Saved 330 fully-paired song names to fully_paired_songs.csv

12. Hand-off: the shared module

All extraction logic and dataset loaders are now packaged in `src/data_utils.py`. Teammates working on the modeling notebooks (Task 1 and Task 2) can `import` from this module and start loading data immediately — they don't need to re-run any of the work above.

The `dataset_summary()` call below is the final smoke test. If it prints sensible numbers, the data pipeline is officially complete and ready to hand off.

```

In [26]: from pathlib import Path

REPO_ROOT = Path.cwd().parent  # synth_music_generator/

data_utils_code = '''
Synth-pop Music Generation – Data Utilities
=====

Shared functions for loading the curated synth-pop MIDI dataset.
Lives at the repo root; reads from ./processed/.

Usage from a notebook in a subfolder (e.g. data_extraction/ or task2/):
    import sys
    from pathlib import Path
    sys.path.insert(0, str(Path.cwd().parent))  # add repo root to path
    from data_utils import dataset_summary, iter_melodies, iter_drums, iter_
'''

import pretty_midi
import numpy as np
import pandas as pd
from pathlib import Path

```



```

# — Configuration —————
# data_utils.py lives at the repo root, so processed/ is right next to it.
PROJECT_ROOT = Path(__file__).parent
PROCESSED_DIR = PROJECT_ROOT / "processed"
MELODY_DIR = PROCESSED_DIR / "melodies_all"
DRUM_DIR = PROCESSED_DIR / "drums_all"
BASS_DIR = PROCESSED_DIR / "bass_all"

# General MIDI Drum Map (channel 9)
GM_DRUM_MAP = {
    35: "Acoustic Bass Drum", 36: "Kick",
    37: "Side Stick", 38: "Snare", 39: "Hand Clap", 40: "Electric Snare",
    41: "Low Floor Tom", 42: "Closed Hi-Hat", 43: "High Floor Tom",
    44: "Pedal Hi-Hat", 45: "Low Tom", 46: "Open Hi-Hat",
    47: "Low-Mid Tom", 48: "Hi-Mid Tom", 49: "Crash Cymbal 1",
    50: "High Tom", 51: "Ride Cymbal 1", 52: "Chinese Cymbal",
    53: "Ride Bell", 54: "Tambourine", 55: "Splash Cymbal",
    56: "Cowbell", 57: "Crash Cymbal 2", 58: "Vibraslap",
    59: "Ride Cymbal 2", 60: "High Bongo", 61: "Low Bongo",
    62: "Mute Hi Conga", 63: "Open Hi Conga", 64: "Low Conga",
    65: "High Timbale", 66: "Low Timbale", 67: "High Agogo",
    68: "Low Agogo", 69: "Cabasa", 70: "Maracas",
    71: "Short Whistle", 72: "Long Whistle", 73: "Short Guiro",
    74: "Long Guiro", 75: "Claves", 76: "High Wood Block",
    77: "Low Wood Block", 78: "Mute Cuica", 79: "Open Cuica",
    80: "Mute Triangle", 81: "Open Triangle",
}

# — Path resolution helper —————

def _resolve(path_value, default_dir):
    """Resolve a path that might be absolute (old) or just a filename (new).
    Always returns a path inside default_dir based on the filename, so this
    works regardless of whose machine generated the CSV."""
    name = Path(str(path_value)).name
    return default_dir / name

# — Metadata loaders —————

def load_metadata():
    """Load full song metadata (BPM, drums, melody info)."""
    return pd.read_csv(PROCESSED_DIR / "song_metadata.csv")

def load_melody_metadata():
    return pd.read_csv(PROCESSED_DIR / "melody_dataset.csv")

def load_drum_metadata():
    return pd.read_csv(PROCESSED_DIR / "drum_dataset.csv")

def load_bass_metadata():

```

```
return pd.read_csv(PROCESSED_DIR / "bass_dataset.csv")

# — MIDI loaders —————

def load_melody_midi(path):
    midi = pretty_midi.PrettyMIDI(str(path))
    return midi.instruments[0] if midi.instruments else None

def load_drum_midi(path):
    midi = pretty_midi.PrettyMIDI(str(path))
    return midi.instruments[0] if midi.instruments else None

def load_bass_midi(path):
    midi = pretty_midi.PrettyMIDI(str(path))
    return midi.instruments[0] if midi.instruments else None

# — Iterators (path-robust) —————

def iter_melodies():
    """Yield (metadata_row, pretty_midi.Instrument) for each melody."""
    df = load_melody_metadata()
    for _, row in df.iterrows():
        try:
            path = _resolve(row["melody_path"], MELODY_DIR)
            if not path.exists():
                continue
            inst = load_melody_midi(path)
            if inst is not None:
                yield row, inst
        except Exception:
            continue

def iter_drums():
    """Yield (metadata_row, pretty_midi.Instrument) for each drum track."""
    df = load_drum_metadata()
    for _, row in df.iterrows():
        try:
            path = _resolve(row["drum_path"], DRUM_DIR)
            if not path.exists():
                continue
            inst = load_drum_midi(path)
            if inst is not None:
                yield row, inst
        except Exception:
            continue

def iter_bass():
    """Yield (metadata_row, pretty_midi.Instrument) for each bass track."""
    df = load_bass_metadata()
    for _, row in df.iterrows():
```

```

        try:
            path = _resolve(row["bass_path"], BASS_DIR)
            if not path.exists():
                continue
            inst = load_bass_midi(path)
            if inst is not None:
                yield row, inst
        except Exception:
            continue

# — Quick Info —————

def dataset_summary():
    """Print a summary of the dataset."""
    mel = load_melody_metadata()
    drm = load_drum_metadata()
    try:
        bas = load_bass_metadata()
        n_bass = len(bas)
    except FileNotFoundError:
        n_bass = 0

    print("=====")
    print("  Synth-pop Dataset")
    print("=====")
    print(f"  Melodies:    {len(mel)} songs")
    print(f"  Drum tracks: {len(drm)} songs")
    print(f"  Bass tracks: {n_bass} songs")
    print(f"  Artists:     {mel['artist'].nunique()}")
    print(f"  BPM range:   {mel['bpm'].min():.0f}–{mel['bpm'].max():.0f}")
    print(f"  Median BPM:  {mel['bpm'].median():.0f}")
    print("=====")
    print("  Top melody instruments:")
    for instr, n in mel["instrument_name"].value_counts().head(5).items():
        print(f"    {instr}: {n}")
    print("=====")

if __name__ == "__main__":
    dataset_summary()
    ...

# Write to repo root
out_path = REPO_ROOT / 'data_utils.py'
with open(out_path, 'w') as f:
    f.write(data_utils_code)

print(f"✓ Wrote data_utils.py to: {out_path}")
print(f"  Size: {out_path.stat().st_size} bytes")

```

✓ Wrote data_utils.py to: /Users/donghyunhahn/Documents/GitHub/synth_music_generator/data_utils.py
 Size: 6772 bytes

```
In [27]: # Confirm data_utils.py landed at the repo root (written by the cell above)
from pathlib import Path

repo_root = Path.cwd().parent
target = repo_root / 'data_utils.py'
print(f"data_utils.py at repo root: {target}")
print(f"  exists: {target.exists()}")
```

```
data_utils.py at repo root: /Users/donghyunhahn/Documents/GitHub/synth_music_generator/data_utils.py
  exists: True
```

```
In [28]: # Test that the shared module works
import importlib
import sys

# Force reload in case it's been imported already this session
if 'data_utils' in sys.modules:
    importlib.reload(sys.modules['data_utils'])

# data_utils.py lives at the repo root (added to sys.path in the setup cell)
# so we import it directly – no 'src.' prefix.
from data_utils import (
    dataset_summary,
    load_melody_metadata,
    load_drum_metadata,
    load_bass_metadata,
    iter_melodies,
    iter_drums,
    iter_bass,
)

# Print summary (now includes bass)
dataset_summary()

# Quick test: load a few of each track type
print("\nFirst 3 melodies:")
for i, (meta, inst) in enumerate(iter_melodies()):
    if i >= 3:
        break
    print(f"  {meta['artist']} – {Path(meta['song']).stem}: "
          f"{meta['instrument_name']}, {len(inst.notes)} notes")

print("\nFirst 3 basslines:")
for i, (meta, inst) in enumerate(iter_bass()):
    if i >= 3:
        break
    print(f"  {meta['artist']} – {Path(meta['song']).stem}: "
          f"{meta['instrument_name']}, {len(inst.notes)} notes")
```

Synth-pop Dataset

Melodies: 335 songs
 Drum tracks: 340 songs
 Bass tracks: 335 songs
 Artists: 27
 BPM range: 70–159
 Median BPM: 120

Top melody instruments:
 Lead 1 (square): 32
 Alto Sax: 25
 Acoustic Grand Piano: 19
 Lead 2 (sawtooth): 18
 Synth Choir: 18

First 3 melodies:

ABC – 4 Ever 2 Gether: Pan Flute, 564 notes
 ABC – Look of Love, Part 1: Synth Brass 1, 350 notes
 ABC – Poison Arrow: Clarinet, 339 notes

First 3 basslines:

ABC – 4 Ever 2 Gether: Fretless Bass, 518 notes
 ABC – Look of Love, Part 1: Synth Bass 2, 590 notes
 ABC – Poison Arrow: Electric Bass (finger), 613 notes

Summary

Pipeline output:

- 335 melody MIDIs across 27 artists, BPM 70–159 (median 120)
- 340 drum MIDIs (97.6% of songs had usable drums)
- Top melody instruments: Lead 1 (square), Alto Sax, Acoustic Grand Piano, Lead 2 (sawtooth), Synth Choir — exactly the synth-pop sound

Key decisions and why:

Decision	Reason
Use LMD Clean MIDI (not LMD-full)	Pre-organized by artist, easier filtering, fewer duplicates
Curated artist list (not genre tags)	LMD doesn't ship with reliable genre metadata; explicit artist matching gives higher precision
Use stored tempo events, not <code>estimate_tempo()</code>	Tempo estimation often returns 2x/0.5x errors
Heuristic melody extraction with score-based ranking	Track naming is inconsistent in LMD; rule-based scoring is more robust than any single rule

Decision	Reason
Merge multi-track drums into one	Some songs split kick/snare across tracks; merging gives a unified drum pattern per song

For the presentation (exploratory analysis section):

- BPM distribution centered exactly at 120 BPM confirms the dataset captures the synth-pop signature
- 97.6% drum coverage justifies Task 2 as feasible
- The extraction pipeline independently rediscovers the canonical synth-pop instrument set, which is implicit validation of the heuristic

Next steps (modeling teammates):

1. Load the melody dataset with `iter_melodies()` and train a melody model (Task 1, unconditioned)
2. Load the drum dataset with `iter_drums()` and train a conditional drum model (Task 2)
3. Combine both at inference to produce a complete generated synth-pop track for the final demo